

Ex. Nos. 4	APPLYING CRUD OPERATIONS
Date: 09-02-2026	

SCENARIO 1: PROJECT TASK MANAGEMENT

The company assigns tasks to employees for various projects. Each task includes the task name, assigned employee, status, and deadline.

1. Create Database and Collections:

Create a MongoDB database called `taskdb`. In this database, create a collection named `tasks` to store task information.

```
> _MONGOSH
> use admin
< switched to db admin
> use taskdb
< switched to db taskdb
> db.createCollection("tasks")
< { ok: 1 }
taskdb> |
```

2. Insert Documents:

Insert a document for a task named "frontend development" assigned to "alice" with a status of "in progress" and a deadline of "2025-03-01".

```
> db.tasks.insertOne({name:"alice",task:"frontend development",status:"in progress",deadline:"2025-03-01"})
< {
  acknowledged: true,
  insertedId: ObjectId('67dc0e86990f9ff08c9059f4')
}
```

3. Update Documents:

a) Change the status of the "frontend development" task to "completed".

```
> db.task.updateOne({task:"frontend development"},{$set:{task:"completed"}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
```

4.

5.

6. b) Extend the deadline of the "frontend development" task to "2025-04-01".

```
> db.task.updateMany({task:"frontend development"},{$set:{deadline:"2025-04-01"}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
```

7.

c) Assign a new employee "bob" to the same task.

8.

9. Read Documents:

Retrieve all tasks that are assigned to "alice" and are marked as "in progress".

10. Delete Documents:

Delete the task document for "frontend development" from the `tasks` collection.

SCENARIO 2: TESTING TEAM MANAGEMENT

The company has a testing team responsible for verifying the functionality of different software products. Each tester is assigned to various test cases, and the status of each test case is tracked (e.g., "passed", "failed", "in progress").

1. Create Database and Collections:

Create a MongoDB database called `testdb`. Inside this database, create two collections: `testers` (for storing tester information) and `testcases` (for storing test case details).

```
> use testdb
< switched to db testdb
```

```
> show dbs
< Classrooms 200.00 KiB
  admin      40.00 KiB
  config     96.00 KiB
  librarydb  168.00 KiB
  local      80.00 KiB
  spacedb    72.00 KiB
  testdb     16.00 KiB
```

```
> db.createCollection("testers")
< { ok: 1 }
> db.createCollection("testcases")
< { ok: 1 }
```

```
> show collections
< testcases
  testers
```

2. Insert Documents:

Insert a document into the **testers** collection for a tester named "alice", with a role of "QA Engineer". Insert a test case document into the **testcases** collection, including the test case name "Login Test", status "in progress", assigned to "alice", and a priority of "high".

```
> db.testers.insertOne({eId:"e101",name:"Alice",roll:"QA Engineer"})
< {
  acknowledged: true,
  insertedId: ObjectId('67dbf183ca5096f1f917488c')
}
```

```
> db.testers.find()
< {
  _id: ObjectId('67dbf183ca5096f1f917488c'),
  eId: 'e101',
  name: 'Alice',
  roll: 'QA Engineer'
}
```

```
> db.testcases.insertOne({tcId:"tc101",tcName:"Login Test",status:"in progress",testerAssigned:"Alice",priority:"High"})
< {
  acknowledged: true,
  insertedId: ObjectId('67dbf245ca5096f1f917488e')
}
```

```
> db.testcases.find()
< {
  _id: ObjectId('67dbf245ca5096f1f917488e'),
  tcId: 'tc101',
  tcName: 'Login Test',
  status: 'passed',
  testerAssigned: 'Alice',
  priority: 'High'
}
```

3. Update Documents:

- a) Change the status of "Login Test" to "passed".

```
> db.testcases.updateOne({tcName:"Login Test"},{$set:{status:"passed"}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

- b) Add a new test case called "Signup Test" with status "failed" and assign it to "bob".

```
> db.testcases.insertOne({tcId:"tc102",tcName:"Sign Up Test",status:"failed",testerAssigned:"Bob",priority:"High"})
< {
  acknowledged: true,
  insertedId: ObjectId('67dbf3deca5096f1f917488f')
}
```

c) Update "alice's" role to "Senior QA Engineer".

```
> db.testers.updateOne({$and:[{eId:"e101"},{name:"Alice"}]},{$set:{roll:"Senior QA Engineer"}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

4. Read Documents:

Retrieve all test cases assigned to "alice" and list their names, statuses, and priorities.

```
> db.testcases.find({testerAssigned:"Alice"},{tcName:1,status:1,priority:1,_id:0})
< {
  tcName: 'Login Test',
  status: 'passed',
  priority: 'High'
}
```

5. Delete Documents:

Delete the "Signup Test" case from the `testcases` collection.

```
> db.testcases.deleteOne({tcName:"Sign Up Test"})
< {
  acknowledged: true,
  deletedCount: 1
}
```

```
> db.testcases.find()
< {
  _id: ObjectId('67dbf245ca5096f1f917488e'),
  tcId: 'tc101',
  tcName: 'Login Test',
  status: 'passed',
  testerAssigned: 'Alice',
  priority: 'High'
}
testdb>|
```

Scenario: Student Management System

The school needs to manage student records, which include the student's name, age, grade, and the courses they are enrolled in. The system allows CRUD (Create, Read, Update, Delete) operations to efficiently manage student data.

1. Create Database and Collection

Create a MongoDB database called `studentdb` to store student information. Inside this database, create a collection called `students`. This collection will contain the following fields for each student:

- `name`: The name of the student (string).
- `age`: The student's age (integer).
- `grade`: The grade the student is in (string, e.g., "10th").
- `courses`: An array of courses the student is enrolled in (array of strings).

2. Insert Document

- Insert a student named "Alice Johnson", who is 16 years old, in the 10th grade, and enrolled in "Math", "Science", and "English".
- Insert another student named "Bob Smith", who is 15 years old, in the 9th grade, and enrolled in "History", "Math", and "Geography"

3. Update Documents

- a) Change Alice Johnson's grade to "11th" after she advances to the next year.
- b) Add a new course, "History," to Alice Johnson's list of enrolled courses.
- c) Change Bob Smith's age to 16 after he celebrates his birthday.

4. Read Documents

- a) Retrieve all students who are enrolled in "Math".
- b) Retrieve all students who are in the 10th grade.
- c) Get all students who are older than 15 years of age.

5. Delete Documents

- a) Delete a specific student, such as "Bob Smith," from the **students** collection.
- b) Remove a course, like "Geography," from Bob Smith's list of courses.

SCENARIO-4

A hospital management system stores patient data in MongoDB.

The system needs to efficiently handle patient records, diagnoses, and treatment history while ensuring scalability and quick retrieval.

(a) Write MongoDB commands to create a new database named "HospitalDB" and a collection "patients".

```
> use HospitalDB
< switched to db HospitalDB
> db["patients"].find()
<
```

```
db.createCollection("patients")
{ ok: 1 }
```

b) Insert at least two sample patient documents with fields: patient_id, name, age, diagnosis, and treatment_plan.


```

db.patients.insertMany([
  {patient_id:"p1",name:"Alex",age:19,diagnosis:"LIVER",treatment_plan:""}
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('67dbbb2e9e04f76e124f13b5')
  }
}
db.patients.insertMany([
  {patient_id:"p2",name:"Ale",age:19,diagnosis:"LIVER",treatment_plan:"done"},
  {patient_id:"p3",name:"Al",age:18,diagnosis:"DIABETES",treatment:"in progress"}
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('67dbbcc99e04f76e124f13b6'),
    '1': ObjectId('67dbbcc99e04f76e124f13b7')
  }
}
}

db.patients.find()
[
  {
    _id: ObjectId('67dbbb2e9e04f76e124f13b5'),
    patient_id: 'p1',
    name: 'Alex',
    age: 19,
    diagnosis: 'LIVER',
    treatment_plan: ''
  },
  {
    _id: ObjectId('67dbbcc99e04f76e124f13b6'),
    patient_id: 'p2',
    name: 'Ale',
    age: 19,
    diagnosis: 'LIVER',
    treatment_plan: 'done'
  },
  {
    _id: ObjectId('67dbbcc99e04f76e124f13b7'),
    patient_id: 'p3',
    name: 'Al',
    age: 18,
    diagnosis: 'DIABETES',
    treatment_plan: 'in progress'
  }
]

```

c) Write a MongoDB query to retrieve all patients diagnosed with "Diabetes" from the "patients" collection.

Modify the query to return only the name and treatment_plan fields while excluding _id.


```

> db.patients.find({diagnosis:"DIABETES"})
< {
  _id: ObjectId('67dbbcc99e04f76e124f13b7'),
  patient_id: 'p3',
  name: 'Al',
  age: 18,
  diagnosis: 'DIABETES',
  treatment_plan: 'in progress'
}
> db.patients.find({diagnosis:"DIABETES"},{name:1,treatment_plan:1,_id:0})
< {
  name: 'Al',
  treatment_plan: 'in progress'
}

```

d) Assume the hospital management decides to archive old patient records. Write a MongoDB query to drop the patient records.

```

> show collections
< patients
> db.patients.drop()
< true

```

A university uses MongoDB to manage its course registration system. The database stores data about students, courses, and registrations. Each course document contains details such as `course_id`, `course_name`, `instructor`, and `enrolled_students`.

Task:

(a) Insert new courses into the courses collection. Provide MongoDB commands to insert at least three courses, each with details for `course_id`, `course_name`, `instructor`, and an empty array for `enrolled_students`.

```
> use mydatabase
< switched to db mydatabase
> db.createCollection("courses")
< { ok: 1 }
```



```
> db.courses.insertMany([
  {course_id:"CS101",course_name:"OS",instructor:"Dr. Smith",enrolled_students:[]},
  {course_id:"CS102",course_name:"MongoDB",instructor:"Dr. Lilly",enrolled_students:[]},
  {course_id:"MA103",course_name:"Probability",instructor:"Dr. Frank",enrolled_students:[]}
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('67dbc3ae66e8ef0c0b83b5d1'),
    '1': ObjectId('67dbc3ae66e8ef0c0b83b5d2'),
    '2': ObjectId('67dbc3ae66e8ef0c0b83b5d3')
  }
}
```



```
db.courses.find()
{
  _id: ObjectId('67dbc3ae66e8ef0c0b83b5d1'),
  course_id: 'CS101',
  course_name: 'OS',
  instructor: 'Dr. Smith',
  enrolled_students: []
}
{
  _id: ObjectId('67dbc3ae66e8ef0c0b83b5d2'),
  course_id: 'CS102',
  course_name: 'MongoDB',
  instructor: 'Dr. Lilly',
  enrolled_students: []
}
{
  _id: ObjectId('67dbc3ae66e8ef0c0b83b5d3'),
  course_id: 'MA103',
  course_name: 'Probability',
  instructor: 'Dr. Frank',
  enrolled_students: []
}
```

(b) Write a MongoDB update operation to enroll students in a course. Assume you need to add students to a course with the `course_id` of "CS101". The students' IDs to add are "S1001", "S1002", and "S1003".

```

> db.courses.updateOne({course_id:"CS101"},{$push:{enrolled_students:{each:["S1001","S1002","S1003"]}}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.courses.find()
< {
  _id: ObjectId('67dbc3ae66e8ef0c0b83b5d1'),
  course_id: 'CS101',
  course_name: 'OS',
  instructor: 'Dr. Smith',
  enrolled_students: [
    'S1001',
    'S1002',
    'S1003'
  ]
}
{
  _id: ObjectId('67dbc3ae66e8ef0c0b83b5d2'),
  course_id: 'CS102',
  course_name: 'MongoDB',
  instructor: 'Dr. Lilly',
  enrolled_students: []
}
{
  _id: ObjectId('67dbc3ae66e8ef0c0b83b5d3'),
  course_id: 'MA103',
  course_name: 'Probability',
  instructor: 'Dr. Frank',
  enrolled_students: []
}

```

(c) Create a MongoDB query to find all courses taught by a specific instructor named "Dr. Smith" and display only the course names and enrolled student IDs.

```

> db.courses.find({instructor:"Dr. Smith"},{_id:0,course_name:1,enrolled_students:1})
< {
  course_name: 'OS',
  enrolled_students: [
    'S1001',
    'S1002',
    'S1003'
  ]
}

```

(d)Also, write a command to delete a course if it has no enrolled students at the end of the registration period.

```

> db.courses.deleteMany({enrolled_students:{$eq:[]}})
< {
  acknowledged: true,
  deletedCount: 2
}
> db.courses.find()
< {
  _id: ObjectId('67dbc3ae66e8ef0c0b83b5d1'),
  course_id: 'CS101',
  course_name: 'OS',
  instructor: 'Dr. Smith',
  enrolled_students: [
    'S1001',
    'S1002',
    'S1003'
  ]
}

```